05. 쉘(Shell)과 프로세스 관리

1. 쉘의 종류와 특징

쉘	개발자	프롬프트	특징
sh (Bourne Shell)	스티븐 본(Stephen Bourne)	`$`	최초의 표준 쉘
csh (C Shell)	빌 조이(Bill Joy)	`%`	C 언어 문법, **히스토리·별칭 기능 최초 도입**
ksh (Korn Shell)	데이비드 콘(David Korn)	`$`	sh 호환 + csh 기능
bash (Bourne Again Shell)	브라이언 폭스(Brian Fox)	`$`	**리눅스 기본 쉘**, sh 호환 + 확장
tcsh	-	`%`	csh 개선판
zsh	폴 팔스타드	`%`	강력한 자동완성
dash	-	`$`	경량, Ubuntu의 `/bin/sh`

출제 포인트

- **리눅스 기본 쉘 = bash, 개발자 = 브라이언 폭스**
- **히스토리·별칭 기능을 처음 도입한 쉘 = csh (빌 조이)**
- root의 기본 쉘도 `/bin/bash`

쉘 관련 명령

```
echo $SHELL            # 현재 로그인 쉘
cat /etc/shells        # 사용 가능한 쉘 목록 ★
chsh                   # 로그인 쉘 변경(대화형)
chsh -s /bin/csh user1 # 특정 쉘로 변경
chsh -l                # 사용 가능 쉘 목록
usermod -s /bin/bash u # 쉘 변경(관리자)
exit / logout          # 쉘 종료
```

> 로그인 쉘 정보는 `/etc/passwd`의 **7번째 필드**에 저장됩니다. 로그인을 막으려면 쉘을 `/sbin/nologin` 또는 `/bin/false`로 지정합니다.

---

2. 쉘 변수와 환경 변수

```
VAR="hello"            # 지역(쉘) 변수 선언 — 공백 없이!
echo $VAR              # 값 출력
export VAR             # 환경 변수로 승격 ★
export VAR="hi"        # 선언과 동시에 환경변수
unset VAR              # 변수 해제
set                    # 모든 쉘 변수 + 함수 목록
env                    # 환경 변수만 목록 ★
printenv               # 환경 변수 출력
printenv PATH          # 특정 환경변수
declare -x VAR         # export와 동일
readonly VAR           # 읽기 전용 변수
```

구분	지역 변수	환경 변수
자식 프로세스 상속	**안 됨**	**됨**
확인 명령	`set`	`env` , `printenv`
선언	`VAR=값`	`export VAR=값`

주요 환경 변수 (암기)

변수	의미
`PATH`	실행 파일 검색 경로 (`:` 로 구분)
`HOME`	홈 디렉터리
`USER` / `LOGNAME`	사용자명
`SHELL`	로그인 쉘
`PWD` / `OLDPWD`	현재 / 이전 디렉터리
`PS1`	**기본 프롬프트 문자열**
`PS2`	2차 프롬프트 (`>`)
`TERM`	터미널 종류
`LANG`	언어·로케일 (`ko_KR.UTF-8`)
`HISTSIZE`	히스토리 저장 개수
`HISTFILE`	히스토리 파일 (`~/.bash_history`)
`HOSTNAME`	호스트명
`MAIL`	메일 저장 경로
`DISPLAY`	X 윈도 디스플레이
`UID` / `EUID`	사용자 ID
`TMOUT`	자동 로그아웃 시간(초)

PS1 프롬프트 특수 문자 (출제 포인트)

기호	의미
`\u`	사용자명
`\h`	호스트명(짧게) / `\H` 전체
`\w`	현재 경로 전체 / `\W` 마지막 디렉터리만
`\d`	날짜 / `\t` 24시간 시간 / `\T` 12시간
`\$`	root면 `#` , 일반 사용자면 `$`
`\n`	줄바꿈
`\!`	히스토리 번호

예: `PS1='[\u@\h \W]\$ '` → `[user1@localhost ~]$`

특수 쉘 변수

변수	의미
`$0`	실행된 스크립트/쉘 이름
`$1` ~ `$9`	위치 인자
`$#`	인자의 개수
`$*` / `$@`	전체 인자
`$?`	**직전 명령의 종료 상태** (0=성공) ★
`$$`	현재 쉘의 PID
`$!`	마지막 백그라운드 프로세스 PID

---

3. 쉘 환경 설정 파일 (읽는 순서 중요)

전역 설정 (모든 사용자)

파일	역할
`/etc/profile`	로그인 시 실행. 환경변수·PATH·umask 설정
`/etc/profile.d/*.sh`	profile이 불러오는 개별 스크립트
`/etc/bashrc` (또는 `/etc/bash.bashrc`)	bash 실행 시마다. 별칭·함수

개인 설정 (사용자별, 홈 디렉터리)

파일	역할
`~/.bash_profile`	**로그인 시 1회** 실행. 환경변수
`~/.bash_login`	bash_profile 없을 때
`~/.profile`	위 둘 없을 때
`~/.bashrc`	**bash 실행할 때마다**. 별칭·함수 ★
`~/.bash_logout`	로그아웃 시 실행
`~/.bash_history`	명령 이력 저장

읽는 순서 (로그인 쉘)

> `/etc/profile` → `/etc/profile.d/*.sh` → `~/.bash_profile` → `~/.bashrc` → `/etc/bashrc`

출제 포인트

- **환경 변수는 `~/.bash_profile`, 별칭(alias)은 `~/.bashrc`**
- 설정 파일 수정 후 즉시 적용: `source ~/.bashrc` 또는 `. ~/.bashrc`

---

4. 명령 이력(History)

```
history                 # 전체 이력
history 10              # 최근 10개
history -c              # 이력 삭제(clear)
history -w              # 메모리 이력을 파일에 기록
!!                      # 직전 명령 재실행 ★
!52                     # 52번 명령 재실행
!ls                     # ls로 시작하는 최근 명령
!$                      # 직전 명령의 마지막 인자
!^                      # 직전 명령의 첫 인자
^old^new                # 직전 명령에서 old를 new로 바꿔 실행
Ctrl+R                  # 역방향 검색
```

관련 환경변수: `HISTSIZE` (메모리 개수), `HISTFILESIZE` (파일 개수), `HISTFILE`, `HISTCONTROL` (ignoredups 등)

---

5. 별칭(alias)과 함수

```
alias                       # 목록
alias ll='ls -alF'          # 설정
alias rm='rm -i'            # 안전장치
unalias ll                  # 해제
\rm file                    # 별칭 무시하고 원래 명령 실행 ★
```

영구 적용: `~/.bashrc` 에 추가

---

6. 쉘 메타문자와 명령 연결

기호	의미
`*`	0개 이상의 모든 문자
`?`	임의의 1문자
`[abc]`	괄호 안 문자 중 하나
`[!abc]` / `[^abc]`	괄호 안 문자 제외
`{a,b}`	중괄호 확장 (`file{1,2}` → file1 file2)
`;`	순차 실행 (결과 무관)
`&&`	앞 명령 **성공** 시에만 뒤 실행
`||`	앞 명령 **실패** 시에만 뒤 실행
`&`	**백그라운드 실행**
`|`	파이프
`` ` ` `` 또는 `$()`	명령 치환
`''`	강한 인용 — 모든 특수문자 무시
`""`	약한 인용 — `$`, `` ` ``, `\` 는 해석됨
`\`	다음 한 문자 이스케이프
`#`	주석

```
echo "오늘은 $(date +%F)"      # 명령 치환
cp file.txt file.txt.bak
mkdir test && cd test          # 성공 시에만 이동
ls nofile ¦¦ echo "없음"        # 실패 시 메시지
```

---

7. 프로세스 개념

용어	의미
PID	프로세스 ID
PPID	부모 프로세스 ID
UID/EUID	실행 사용자 ID
데몬(Daemon)	백그라운드 상주 서비스 프로세스 (`httpd` , `sshd`)
포그라운드	터미널을 점유하고 실행
백그라운드	터미널과 무관하게 실행 (`&`)
고아 프로세스(Orphan)	부모가 먼저 종료된 프로세스 → init/systemd가 인수
좀비 프로세스(Zombie)	**종료됐지만 부모가 종료 상태를 회수하지 않은** 프로세스. 상태 `Z` , `defunct` 표시

PID 1

- SysV init 시스템: `init`
- systemd 시스템: `systemd`

프로세스 생성 방식 (출제 포인트)

방식	설명
fork	부모 프로세스를 **복제**해 자식 생성 (새 PID 부여)
exec	**현재 프로세스를 새 프로그램으로 덮어씀** (PID 유지)

데몬 실행 방식

방식	설명
standalone	부팅 시 상주하며 직접 요청 처리. 빠름. (`httpd` , `sshd`)
inetd / xinetd (슈퍼 데몬)	요청이 올 때만 해당 데몬 실행. 메모리 절약. 설정: `/etc/xinetd.conf` , `/etc/xinetd.d/`

---

8. 프로세스 확인 — ps / top

```
ps                    # 현재 터미널의 프로세스
ps -e                 # 모든 프로세스 (every)
ps -f                 # 상세 정보 (full)
ps -ef                # ★ 가장 많이 쓰는 조합 (System V 방식)
ps aux                # ★ BSD 방식 (a=모든터미널, u=사용자정보, x=터미널없는것도)
ps -ef | grep httpd   # 특정 프로세스 검색
ps -u user1           # 특정 사용자의 프로세스
ps -l                 # 긴 형식
ps -eo pid,ppid,cmd   # 출력 필드 지정
ps --forest           # 트리 형태
pstree                # 프로세스 계층 트리 ★
pstree -p             # PID 포함
pgrep httpd           # 이름으로 PID 검색
pidof httpd           # 이름으로 PID 검색
```

`ps aux` vs `ps -ef` (혼동 주의)

- `aux` = **BSD 스타일** (하이픈 없음)
- `-ef` = **System V 스타일** (하이픈 있음)
- 둘 다 "모든 프로세스 상세 출력"이라는 목적은 동일

`ps -ef` 출력 읽는 법 (네트워크관리사 2026-3회 41번)

```
UID    PID  PPID  C STIME  TTY      TIME     CMD
icqa  1868     1  0 8월25  ?     00:00:00 /usr/sbin/sshd
icqa  1869  1868  0 8월25  ?     00:00:00 /bin/sh -c /home/icqa/project/venv/bin/python ...
```

열	의미
UID	프로세스를 실행한 사용자
PID	프로세스 ID
PPID	**부모** 프로세스 ID (부모 PID)
C	CPU 사용률(스케줄링용)
STIME	시작 시각
TTY	제어 터미널 (`?` = 터미널 없음, 데몬 등)
TIME	누적 CPU 사용 시간
CMD	실행 명령

> 읽기 요령: **같은 행의 PID와 PPID를 혼동하지 마세요.** 위 예에서 두 번째 행의 PID는 1869이고 PPID는 1868입니다. 프로세스 종료는 **해당 행의 PID**로 `kill PID` 입니다 (PPID가 아님). 기출 41번: '네 번째 프로세스의 PPID는 1869'(PID와 혼동)가 옳지 않은 설명이었습니다. `ps -ef | grep 패턴` 으로 검색하면 `grep` 자신의 행도 함께 나올 수 있습니다.

프로세스 상태 코드 (STAT)

코드	의미
`R`	Running — 실행 중
`S`	Sleeping — 인터럽트 가능 대기

코드	의미
`D`	Uninterruptible sleep — I/O 대기
`T`	Stopped — 중지
`Z`	**Zombie — 좀비**
`<`	높은 우선순위
`N`	낮은 우선순위
`s`	세션 리더
`+`	포그라운드 프로세스 그룹
`l`	멀티스레드

top — 실시간 모니터링

```
top                 # 실행
top -d 3            # 3초 간격 갱신
top -u user1        # 특정 사용자만
top -b -n 1         # 배치 모드(스크립트용)
```

top 내부 대화형 키

키	기능
`h`	도움말
`k`	**프로세스 종료(kill)**
`r`	**우선순위 변경(renice)**
`M`	메모리 사용량 순 정렬
`P`	CPU 사용량 순 정렬
`T`	실행 시간 순 정렬
`u`	특정 사용자 필터
`1`	CPU 코어별 표시
`c`	전체 명령줄 표시
`q`	종료
`space`	즉시 갱신

기타 모니터링

```
uptime              # 부하 평균(load average) 1분/5분/15분
vmstat 1            # 가상 메모리 통계
iostat              # 디스크 I/O
sar                 # 시스템 활동 보고 (sysstat 패키지)
free -h             # 메모리
lsof                # 열린 파일 목록
lsof -i :80         # 80번 포트 사용 프로세스 ★
lsof -u user1       # 사용자별
fuser -v /mnt       # 특정 파일/디렉터리 사용 프로세스
```

---

9. 프로세스 종료 — kill / signal

```
kill PID              # 기본 TERM(15) 시그널
kill -9 PID           # 강제 종료 (KILL)
kill -15 PID          # 정상 종료 요청 (TERM)
kill -l               # 시그널 목록 확인 ★
kill -HUP PID         # 설정 재읽기
killall httpd         # 이름으로 모두 종료
pkill -u user1        # 사용자의 프로세스 종료
pkill -f "pattern"    # 명령줄 패턴으로 종료
```

주요 시그널 (번호 암기 ★★★)

번호	이름	의미
1	**SIGHUP**	터미널 연결 종료 / **데몬 설정 재읽기**
2	**SIGINT**	인터럽트 — **Ctrl + C**
3	SIGQUIT	종료 + 코어 덤프 — Ctrl + \
9	**SIGKILL**	**강제 종료 (무시 불가)**
11	SIGSEGV	세그먼테이션 오류
15	**SIGTERM**	**정상 종료 요청 (kill 기본값)**
18	SIGCONT	중지된 프로세스 계속 실행
19	**SIGSTOP**	프로세스 중지 (무시 불가)
20	SIGTSTP	터미널에서 중지 — **Ctrl + Z**

출제 포인트

- `kill`의 **기본 시그널은 15(SIGTERM)**, `-9`가 아님
- **9(SIGKILL)와 19(SIGSTOP)는 무시하거나 가로챌 수 없음**
- Ctrl+C = 2(INT), Ctrl+Z = 20(TSTP)

---

10. 작업 제어(Job Control)

```
command &             # 백그라운드로 실행
jobs                  # 작업 목록 (작업번호 확인)
jobs -l               # PID 포함
fg                    # 가장 최근 작업을 포그라운드로
fg %1                 # 1번 작업을 포그라운드로 ★
bg %1                 # 1번 작업을 백그라운드에서 계속
Ctrl + Z              # 현재 작업 중지(suspend)
Ctrl + C              # 현재 작업 강제 종료
nohup command &       # 로그아웃 후에도 계속 실행 ★ (출력은 nohup.out)
disown %1             # 쉘 종료와 작업 연결 해제
```

작업 번호 표기

기호	의미
`%n`	n번 작업
`%+` 또는 `%%`	현재 작업
`%-`	이전 작업

`jobs` 출력의 `+`는 현재 작업, `-`는 이전 작업입니다.

---

11. 프로세스 우선순위 — nice / renice

- **NI (nice 값) 범위: -20 ~ 19**
- **값이 작을수록 우선순위 높음** (-20이 최우선)
- 기본값: **0**
- **우선순위를 높이는 것(음수 지정)은 root만 가능**

```
nice -n 10 command        # nice 값 10으로 실행
nice -10 command          # 동일
nice --10 command         # nice 값 -10 (root만)
renice 5 -p 1234          # 실행 중인 PID의 nice 값 변경
renice -n 5 -p 1234       # 동일
renice 5 -u user1         # 사용자의 모든 프로세스
renice -5 -p 1234         # 우선순위 높이기(root만)
ps -l                     # PRI, NI 열로 확인
```

열	의미
PRI	커널이 계산한 실제 우선순위 (낮을수록 높음)
NI	사용자가 지정한 nice 값

---

12. 작업 예약 — cron / at

cron — 반복 예약

```
crontab -e            # 편집 ★
crontab -l            # 목록 확인
crontab -r            # 전체 삭제 (주의!)
crontab -u user1 -l   # 특정 사용자 확인(root)
```

crontab 형식 (필드 5개 — 반드시 암기)

```
분  시  일  월  요일  실행할_명령
*   *   *   *   *
|   |   |   |   └──── 요일 (0~7, 0과 7=일요일, 1=월요일)
|   |   |   └──────── 월   (1~12)
|   |   └──────────── 일   (1~31)
|   └──────────────── 시   (0~23)
└──────────────────── 분   (0~59)
```

기호	의미
`*`	모든 값
`,`	목록 (`1,15`)
`-`	범위 (`1-5`)
`/`	간격 (`*/10` = 10마다)

예제 (시험 단골)

표현	의미
`0 3 * * *`	매일 새벽 3시 0분
`30 2 1 * *`	매월 1일 02:30
`*/10 * * * *`	10분마다
`0 */2 * * *`	2시간마다 정각
`0 9 * * 1`	매주 월요일 09:00
`0 0 * * 0`	매주 일요일 자정
`15 14 1 * *`	매월 1일 14:15
`0 22 * * 1-5`	평일 22:00
`5 4 * * sun`	매주 일요일 04:05

cron 관련 파일

파일/디렉터리	역할
`/etc/crontab`	시스템 crontab (**사용자 필드 추가로 6번째 필드에 실행 계정**)
`/etc/cron.d/`	추가 cron 설정
`/var/spool/cron/`	사용자별 crontab 저장 위치
`/etc/cron.allow`	사용 허용 사용자 목록
`/etc/cron.deny`	사용 금지 사용자 목록
`/etc/cron.hourly` `.daily` `.weekly` `.monthly`	주기별 스크립트 디렉터리
데몬	`crond`

> ⚠️ **`/etc/crontab` 의 필드는 6개**(분 시 일 월 요일 **사용자** 명령), 사용자 crontab은 5개입니다. 함정 문제로 나옵니다.

at — 1회성 예약

```
at 10:00                    # 오늘 10시 예약(대화형, Ctrl+D로 종료)
at 5pm tomorrow
at now + 2 hours
at 10:00 2026-12-12
atq                         # 예약 목록 (= at -l)
atrm 3                      # 3번 작업 삭제 (= at -d 3)
at -f script.sh 10:00       # 파일로 지정
```

관련 파일: `/etc/at.allow` , `/etc/at.deny` , 데몬 `atd` , 스풀 `/var/spool/at`

cron vs at

구분	cron	at
실행	**반복**	**1회성**
데몬	`crond`	`atd`
목록	`crontab -l`	`atq`
삭제	`crontab -r`	`atrm`

---

13. systemd 서비스 관리 (현대 리눅스 필수)

```
systemctl start httpd           # 시작
systemctl stop httpd            # 정지
systemctl restart httpd         # 재시작
systemctl reload httpd          # 설정만 다시 읽기
systemctl status httpd          # 상태 확인 ★
systemctl enable httpd          # 부팅 시 자동 시작 등록 ★
systemctl disable httpd         # 자동 시작 해제
systemctl is-enabled httpd      # 자동시작 여부
systemctl is-active httpd       # 동작 여부
systemctl list-units            # 유닛 목록
systemctl list-unit-files       # 유닛 파일 목록
systemctl mask httpd            # 완전 차단(시작 불가)
systemctl daemon-reload         # 유닛 파일 변경 반영
journalctl -u httpd             # 서비스 로그
journalctl -f                   # 실시간 로그
journalctl -b                   # 현재 부팅 이후 로그
```

구 방식 대응표

SysV	systemd
`service httpd start`	`systemctl start httpd`
`chkconfig httpd on`	`systemctl enable httpd`
`chkconfig --list`	`systemctl list-unit-files`

유닛 파일 위치: `/usr/lib/systemd/system/` (패키지 제공), `/etc/systemd/system/` (관리자 정의, 우선)

---

14. 마무리 체크 문제

1. 리눅스 기본 쉘과 개발자? → **bash / 브라이언 폭스**
2. 히스토리·별칭을 처음 도입한 쉘? → **csh (빌 조이)**
3. 사용 가능한 쉘 목록 파일? → `/etc/shells`
4. 환경변수는 어느 파일에, 별칭은 어느 파일에? → `~/.bash_profile` / `~/.bashrc`
5. 직전 명령의 성공 여부를 담은 변수? → `$?`
6. `kill` 의 기본 시그널 번호와 이름? → **15, SIGTERM**
7. 무시할 수 없는 시그널 2개? → **9(SIGKILL), 19(SIGSTOP)**
8. Ctrl+Z가 보내는 시그널? → **20 (SIGTSTP)**
9. 좀비 프로세스의 STAT 코드? → **Z**

10. nice 값의 범위와 최우선 값? → **-20 ~ 19, -20**
11. 우선순위를 높이려면 누구여야 하는가? → **root**
12. 로그아웃 후에도 실행을 유지하는 명령? → `nohup`
13. `30 4 * * 0` 의 의미? → **매주 일요일 04:30**
14. 10분마다 실행하는 crontab 표현? → `*/10 * * * *`
15. `/etc/crontab`의 필드 개수와 6번째 필드? → **6개, 실행 사용자**
16. 1회성 예약 명령과 목록 확인 명령? → `at` / `atq`
17. 부팅 시 서비스 자동 시작을 등록하는 systemd 명령? → `systemctl enable`
18. `fork`와 `exec`의 차이? → fork는 복제(새 PID), exec는 현재 프로세스 덮어쓰기(PID 유지)